LABORATORY 7: Introduction to computer vision

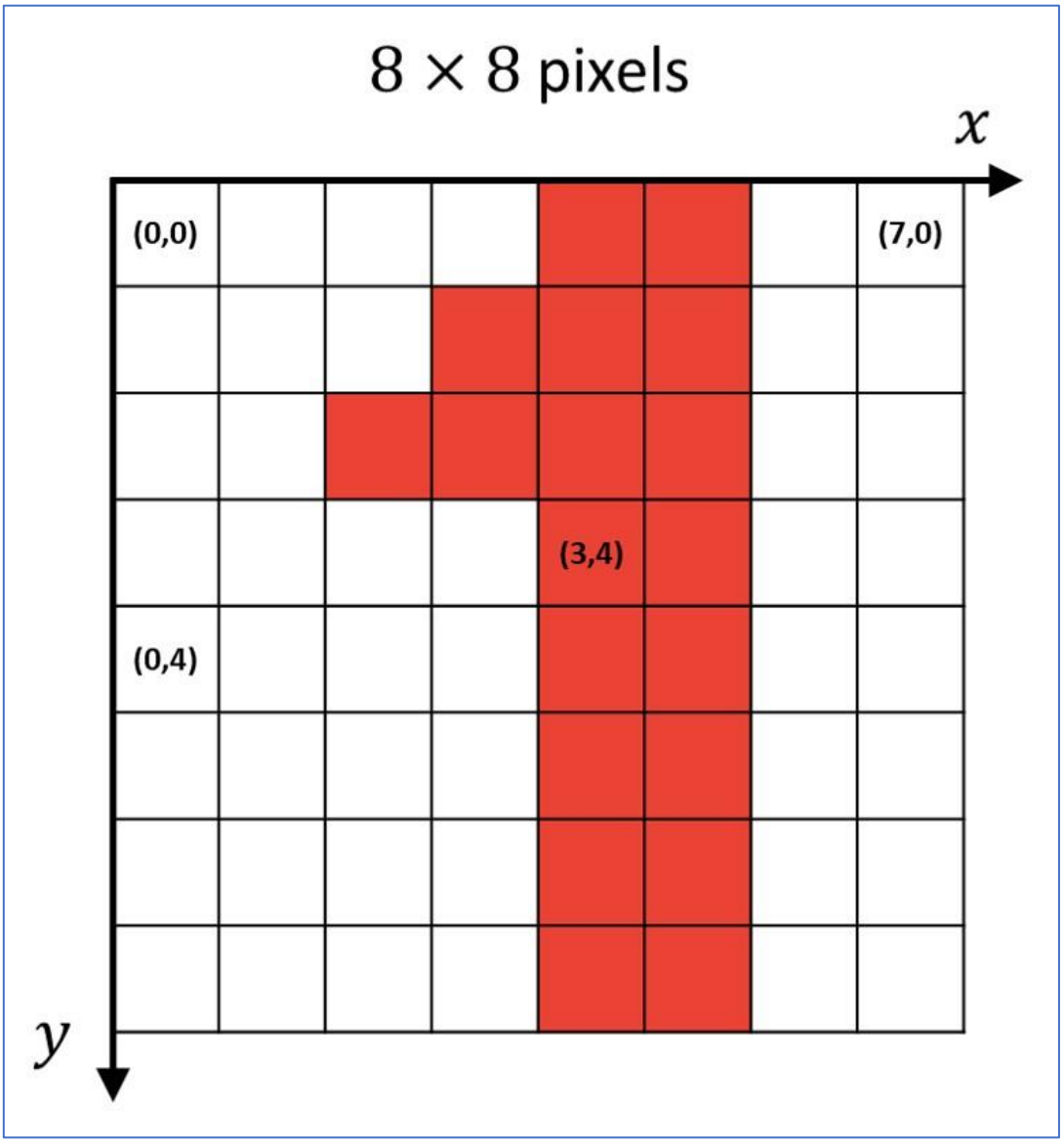
Part 1: Computer Vision.

Theory Concepts:

Computer vision allows different systems to process images and video with different purposes. There are a lot of applications for embedded systems that use computer vision to generate a response or interact with environment changes.

- In your lab inform mention 2 projects that use embedded computer vision.

One of the most used libraries to work with embedded systems is OpenCV. This library can be found in different programming languages and eases the different pixel transformations a developer needs to implement for a project.



As we can see in the image every image has a different quantity of pixels, this determines the size of the image and works as a matrix that can be affected by different operations.

- What is a pixel? Why do we use it to measure image resolutions?
- What values can be assigned to a pixel? What determine a pixel value?

To work with OpenCV we will use module import in python. This is a common technique used to modularize large projects and avoid high coupling with low cohesion.

Forst of all we need to install opencv, for that we can check the command required in the OpenCV official webpage:

https://opencv.org/get-started/

Now to use OpenCV we will:

- Import the library
- Use the imread() function
- Make image processing
- Use the imshow() function

Let's review the following example:

```python
import cv2


def resize_img(img, width, height):
    up_points = (width, height)
    img_resize = cv2.resize(img, up_points)
    return img_resize


if __name__ == "__main__":
    img = cv2.imread("assets\img1.jpg")
    rimg = resize_img(img, 500, 500)
    cv2.imshow("resize image", rimg)
    cv2.waitKey(0)
```

- How does this code work?
- What does __name__=="__main__" do?

To work properly with OpenCV is important to use good practices related to python programming and implement them to improve our code.

Exercises:

1. **Create a program that open an image and resizes it to 1000x1000 pixels.**
2. **Generate a program that asks the user to select between original size, small, medium, or big image size and re scale an image.**

3. **Create a program that rotates 90 degrees an image every time a user presses any keyboard key. Which is the difference between printing an image and using imshow**
4. **Using OpenCV read an image and do the following operations:**
   • Read the image and show it.
   • Resize the image to 400x600 (width, height).
   • Cut the image horizontally and show the two halves.
   • Cut the image vertically and show the two halves.
   • Divide the image in quadrants of the same size and show them with the title "Quadrant N" according to the number of quadrants.

With OpenCV we can also open our webcam and also videos stored in our system, for that we will use the function 'VideoCapture()'. For this function we can indicate a number (0 for default camera) or a video file location, let's see the following example:

```python
import cv2
from abc import ABC, abstractmethod


class video_Capture_abs(ABC):
    @abstractmethod
    def display_camera(self):
        pass

    @abstractmethod
    def stop_display(self):
        pass

    @abstractmethod
    def camera_visualization(self):
        pass
```

```python
class video_Capture(video_Capture_abs):
    def __init__(self, camera) -> None:
        self.camera = camera
        self.displayed = False

    def display_camera(self):
        self.displayed = True
        self.camera_visualization()

    def stop_display(self):
        self.displayed = False

    def camera_visualization(self):
        while self.displayed:
            check, frame = self.camera.read()
            cv2.imshow("camera", frame)
            key = cv2.waitKey(1)
            if key == 27:  # 27 is the ASCII code for the 'Esc' key
                self.stop_display()


if __name__ == "__main__":
    camera = cv2.VideoCapture(0)
    camera_object = video_Capture(camera)
    camera_object.display_camera()
```

As we can see VideoCapture allows us to select the camera. However, to get actual frame information we need to use the .read() function in the camera object.

- Which OOP pillars were used in the previous code? Why?
- What value gets the ‘check’ variable from ‘camera_visualization’ method?
- Investigate how to change the color filter for an image and a video using OpenCV.

Exercises:

1. **Create a folder named “colors” with 3 images of 3 different colors. Then using OpenCV open each image and print the color value on terminal.**
2. **From the previous exercise make a function that converts rgb color to grayscale.**
3. **Escalate the previous exercise with a class that can modify the original color from an image to RGB, HSV and grayscale. Then show the image modified. What is HSV?**

4. **Using HSV detect red color and show it in console, then green and blue. How can you pass this values to TIVA?**
5. **Open a webcam using OpenCV and show the frame in grayscale and then in RGB. Change between grayscale and RGB using keyboard keys.**
6. **Investigate how to capture a frame image and save it as an image using OpenCV. Then, capture the image of the webcam and save the frame every time the key "c" is pressed with the format "Captures/image1.jpg". When the process is terminated, apply a grayscale filter to the image and divide it in quadrants.**